• PYTHON 语言与深度学习 · 课程设计

Python 语言与深度学习

课程设计汇报

五个独立项目的技术方案、实现方法与实验结果。
涵盖强化学习、信号分类、Web 服务及桌面应用。

小组	Coco
汇报人	林子文，许睿泽
组员	万桦平，吴文涛

开始项目演示 ↘

PROJECT DIRECTORY 05 / PROJECTS

01 强化学习贪吃蛇 Reinforcement learning ↗
02 雷达调制识别 Signal classification ↗
03 SimpleChat Online communication ↗
04 ffind 文件查找器 Local search engine ↗
05 SimplePlayer Desktop media player ↗

05 个项目 / 04 个 Web 界面 / 03 个公网演示

向下滚动，进入真实项目 ↓

---

01 / REINFORCEMENT LEARNING

• 公网交互演示

基于强化学习的贪吃蛇系统

用 DQN 学习动作的长期收益，再由安全规划筛选可行方向，每走一格重新判断。

PyTorch　Double DQN　FastAPI / WebSocket

决策原理

神经网络估计收益 · 路径规划筛选候选

01 状态表示与价值网络

状态信息	维度	含义
食物相对位置	2	归一化的横向、纵向距离
四向危险	4	上、下、左、右是否会碰撞
当前朝向	4	用独热编码表示移动方向

10 状态 → 64 隐藏层 → 64 隐藏层 → 4 方向 Q 值

`Q(s, a)` 估计在状态 s 采取动作 a 后的累计折扣奖励。吃到食物得正奖励，碰撞受到惩罚，模型由此学习方向的长期价值。

02 预判规划与动作筛选

01 **吃食预判**

用时间感知 BFS 寻找食物路线，考虑蛇尾逐步让路。让虚拟蛇试走，检查吃完后是否仍能通向蛇尾；蛇较长时还检查空白区域连通性。

02 **追尾周旋**

直接吃食不安全时，优先保留能通向蛇尾、可活动空间较完整的方向，等待身体移动后释放通道。

03 **哈密顿锁定**

蛇较长或长时间未吃到食物时，尝试构建覆盖棋盘的闭合环。成功后保持蛇身沿环的顺序，并在安全范围内寻找捷径。

每步重新决策　读取状态 → 规划器给出候选方向 → 选择候选中 Q 值最大的动作 → 移动一格

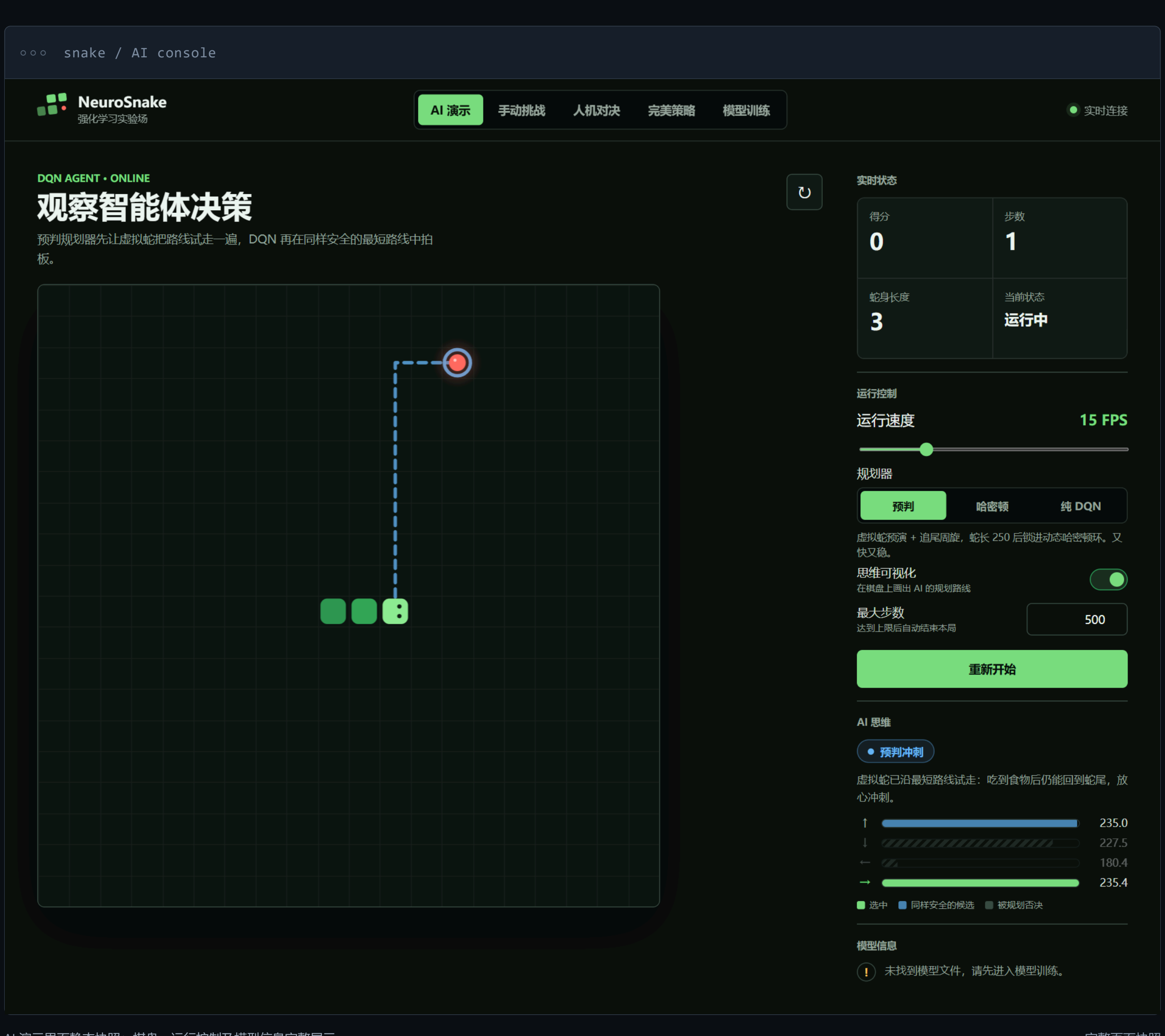
训练阶段 · 学习价值
经验回放 + Double DQN
存储"状态、动作、奖励、新状态"，随机抽取经验训练。在线网络选动作，目标网络估值，以 Huber 损失修正预测，减少连续样本相关性与价值高估。

演示阶段 · 使用权重
安全候选筛选 + 最大 Q 值选择
加载已训练权重，关闭随机探索，也不再更新网络。多个候选时由 DQN 排序；只剩一个候选则直接执行，下一格继续判断。

> **模型与规划器的分工**
> DQN 只接收 10 维局部状态，全局路线由规划器读取完整棋盘计算。默认表现来自二者配合；关闭规划仍保留单步合法动作过滤。Q 值是收益估计，不是成功概率。

环境：20 × 20 棋盘 / 4 个方向动作

纯 DQN、预判规划与完美策略可在原版界面中对照。



观察智能体决策

预判规划器先让虚拟蛇把路线试走一遍，DQN 再在同样安全的最短路线中拍板。

得分 0　步数 1　蛇身长度 3　当前状态 运行中

运行速度 15 FPS

虚拟蛇预演 + 追尾周旋，蛇长 250 后锁进动态哈密顿环。又快又稳。

思维可视化：在棋盘上画出 AI 的规划路线

最大步数：达到上限后自动结束本局　500

虚拟蛇已沿最短路线试走：吃到食物后仍能回到蛇尾，放心冲刺。

↑ 235.0　↓ 227.5　← 180.4　→ 235.4

未找到模型文件，请先进入模型训练。

AI 演示界面静态快照；棋盘、运行控制及模型信息完整展示。

完整页面快照

---

02 / SIGNAL CLASSIFICATION

• 公网交互演示

雷达脉内调制样式识别

将 IQ 脉冲转换为时频图，利用卷积网络学习频率轨迹与调制类别之间的对应关系。

PyTorch　NumPy / h5py　STFT-CNN　Flask

识别原理

复数信号 → 时频表示 → 卷积分类

01 IQ 信号与时频变换

$$x[n] = I[n] + jQ[n]$$

I、Q 为相互正交的两个分量，共同表示信号的幅度与相位。

处理步骤	实现参数	作用
分帧加窗	Hann 窗 64 点 步长 32 点	在滑动的短时间段内分析信号
傅里叶变换	64 点 FFT	得到每个时间段的频率分量
功率映射	$\lvert STFT\rvert^2$ → dB	将各频率的能量分布转成图像强度

按每帧最大功率映射 40 dB 动态范围，再缩放为 96 × 96。时频图横轴为时间、纵轴为频率，亮度表示相对功率，保留频率随时间变化的轨迹。

02 卷积特征与类别输出

时频图 → 卷积 / ReLU / 池化 × 2 → 全连接

卷积提取局部纹理，池化汇总邻域特征，自适应池化统一到 7 × 7 后送入全连接层。模型从带标签的样本中学习纹理与类别的对应关系。

信号示意	时频轨迹
CW 单频	近似水平的频率线
LFM 线性调频	上升或下降的斜线
FSK 频移键控	在离散频率间跳变

输出各类别得分，经 Softmax 转为相对分数，取最高分对应的调制样式。以上轨迹用于解释特征，实际分类由训练后的网络完成。

单脉冲识别　读取 IQ → 校验长度与采样率 → 生成 STFT 功率谱 → CNN 推理 → 返回类别及各类分数

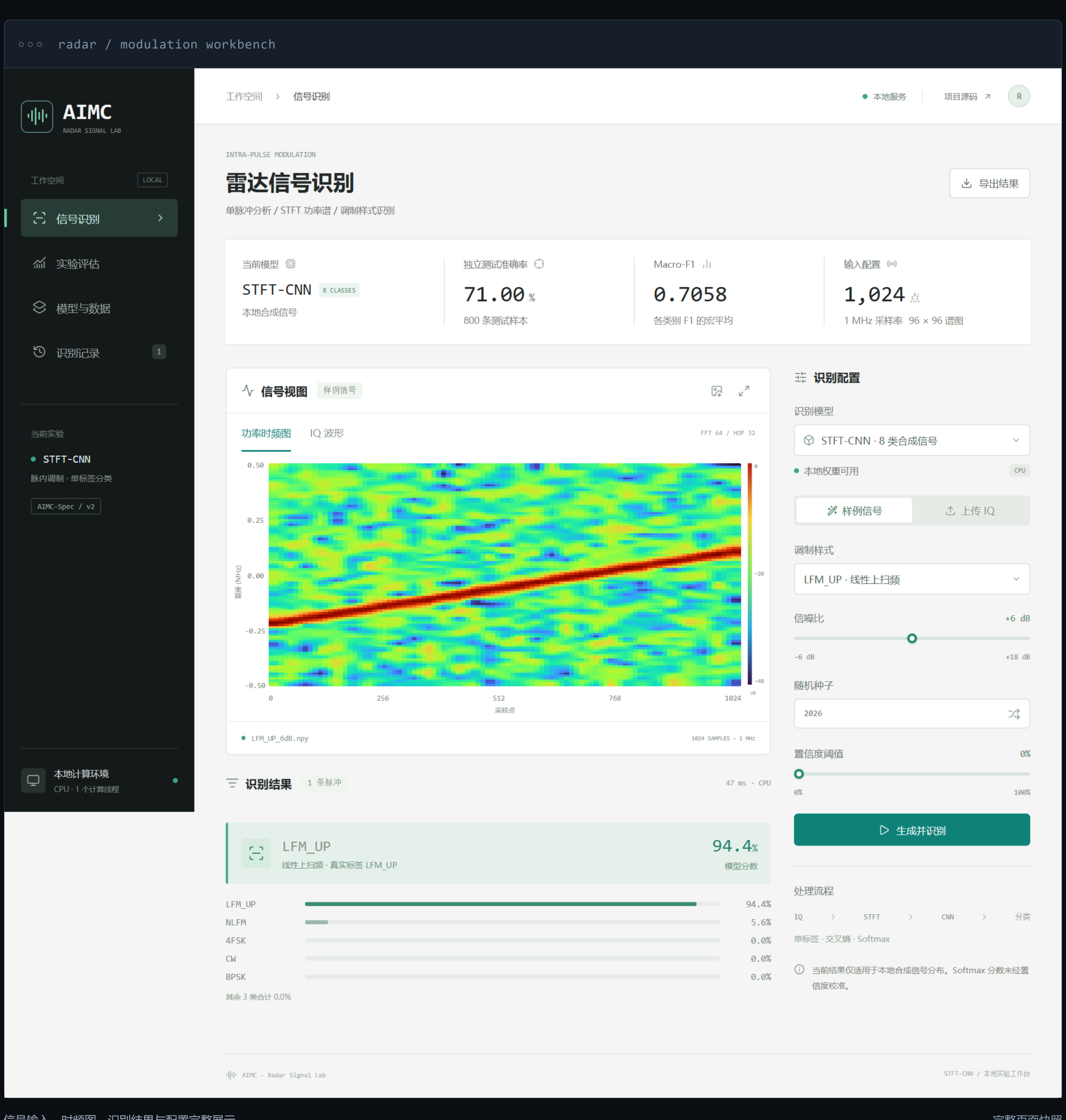
训练阶段 · 有标签监督
时频图 + 真实类别 → 交叉熵损失
通过反向传播调整卷积与分类层参数，按验证集损失保存权重，最后用留出的测试集评估。合成模型与官方模型使用不同的类别集合和输入配置。

识别阶段 · 固定预处理
8 类：1024 点 / 1 MHz
30 类：1000 点 / 50 MHz
加载对应权重，保持训练与推理的预处理一致。输入格式统一为 I、Q 两通道；拒绝全零、非有限值或长度不匹配的数据，不在识别时继续训练。

> **识别结果的适用范围**
> 8 类合成测试准确率为 71.00%，官方 30 类小样本为 7.61%，后者仅用于流程演示。Softmax 分数未经置信度校准，最高分不代表一定正确，也不能据此保证未知信号可被可靠识别。

输入：.npy / .npz / 无表头两列 I,Q 的 .csv

STFT 参数相同；两种模型的输入配置及附加预处理分别随权重加载。



雷达信号识别

单脉冲分析 / STFT 功率谱 / 调制样式识别

当前模型 STFT-CNN 8 CLASSES 本地合成信号

独立测试准确率 71.00% 800 条测试样本

Macro-F1 0.7058 各类别 F1 的宏平均

输入配置 1,024 点 1 MHz 采样率 96 × 96 谱图

STFT-CNN · 脉内调制 · 单标签分类

LFM_UP 94.4% 线性上扫频 · 真实标签 LFM_UP 模型分数

LFM_UP	94.4%
NLFM	5.6%
4FSK	0.0%
CW	0.0%
BPSK	0.0%

其余 3 类合计 0.0%

识别模型：STFT-CNN · 8 类合成信号
本地权重可用 CPU
调制样式：LFM_UP · 线性上扫频
信噪比：+6 dB（-6 dB … +18 dB）
随机种子：2026
置信度阈值：0%（0% … 100%）

当前结果仅适用于本地合成信号分布。Softmax 分数未经置信度校准。

信号输入、时频图、识别结果与配置完整展示。

完整页面快照

---

03 / HTTP COMMUNICATION

• 公网交互演示

SimpleChat 在线聊天室

面向反向代理与单端口部署，采用标准库 HTTP 服务与增量消息同步。

Python 标准库　http.server　原生 JavaScript　JSON API

通信结构

普通 HTTP 请求 / 同源页面与接口

JS **浏览器前端** 房间列表 / 密码加入 / 成员管理

每 1 秒携带 since 游标请求 ⇄ 只返回新增消息与最新状态

Py **Python HTTP 服务** 静态页面 + JSON API / 单端口托管

同步正确性

三个不变量

01 **请求串行化**
pollBusy 保证同一时刻只有一个请求在途，pollAgain 记录需要补拉的状态。

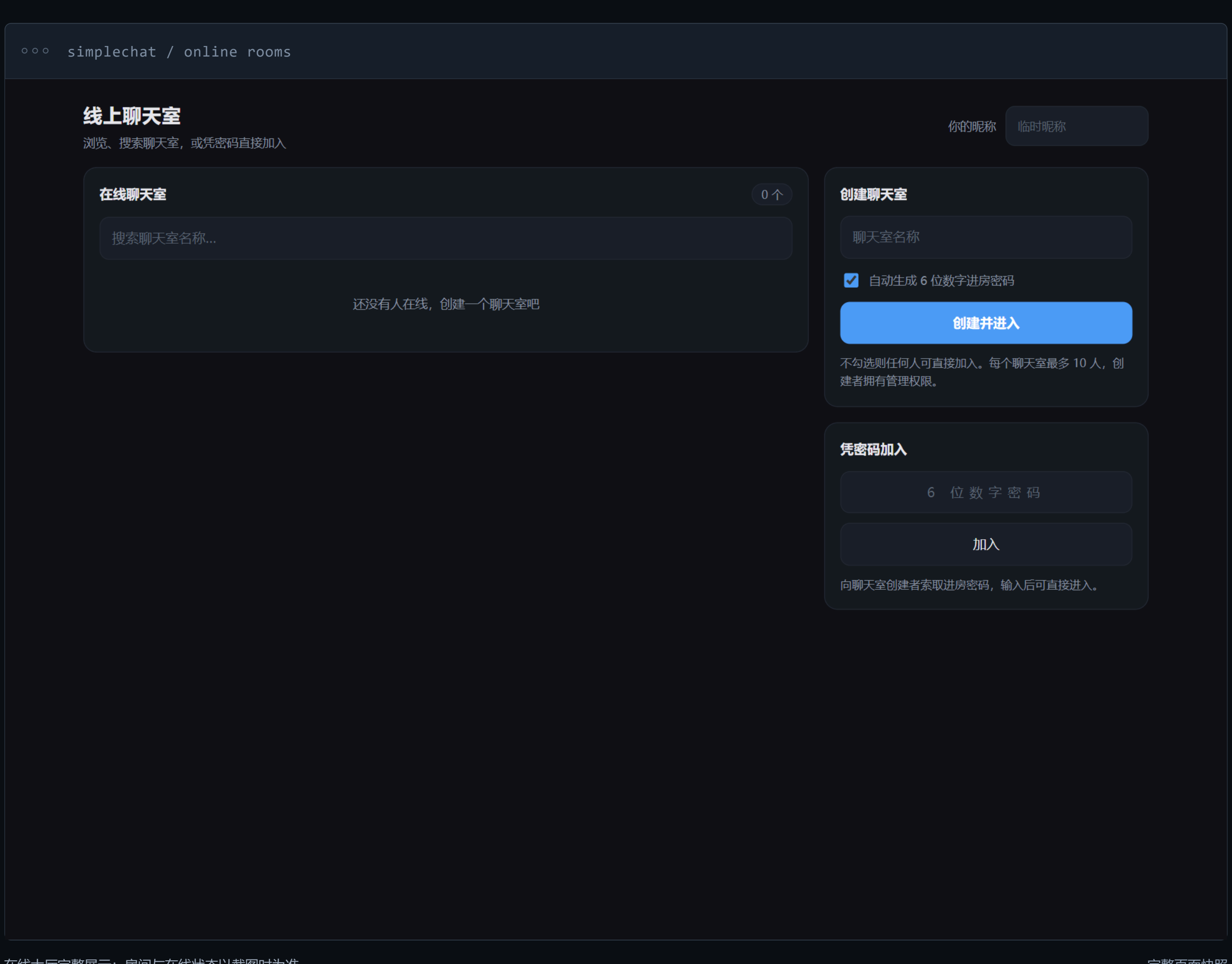
02 **消息幂等**
维护已渲染消息 ID 集合，同一批响应重复到达时只追加一次。

03 **会话隔离**
响应先比对 owner 引用，房间切换后整包丢弃旧会话的过期数据。

12 HTTP 接口 | 10 人 单房间上限 | 15 s 心跳超时 | 200 条 单房间保留消息

鉴权 token 与公开成员 mid 分离

长连接支持无法预先确认，因此采用普通 HTTP 短轮询。



线上聊天室

浏览、搜索聊天室，或凭密码直接加入

还没有人在线，创建一个聊天室吧

☑ 自动生成 6 位数字进房密码

不勾选则任何人可直接加入。每个聊天室最多 10 人，创建者拥有管理权限。

向聊天室创建者索取进房密码，输入后可直接进入。

在线大厅完整展示；房间与在线状态以截图时为准。

完整页面快照

---

04 / INDEXED FILE SEARCH

• 本地 Web 应用

基于 SQLite 索引的文件查找器

以完整路径组织文件记录，通过文件名联合索引加速检索，数据保存在本地 SQLite 数据库中。

SQLite B-tree　WITHOUT ROWID　联合索引

索引结构

一张文件表 + 一个文件名联合索引

01 文件表 `files`

字段	类型	作用
`path` 主键	TEXT	完整路径，区分同名文件
`name`	TEXT	文件名，使用 NOCASE 比较规则
`is_dir`	INTEGER	0 为文件，1 为目录

`WITHOUT ROWID` 直接以 `path` 主键组织表数据，省去额外的隐藏行号结构。NOCASE 默认处理 ASCII 大小写。

02 联合索引 `files_name`

```
CREATE INDEX files_name
ON files(name, path);
```

第一排序键 `name` → 同名时按 `path` 排序

排序示意

notes.txt	…/docs/notes.txt
report.pdf	…/course/report.pdf
report.pdf	…/work/report.pdf
report_1.pdf	…/work/report_1.pdf

`meta(key, value)`　元信息表：记录索引根目录、排除目录、构建时间，以及文件与目录条目数。

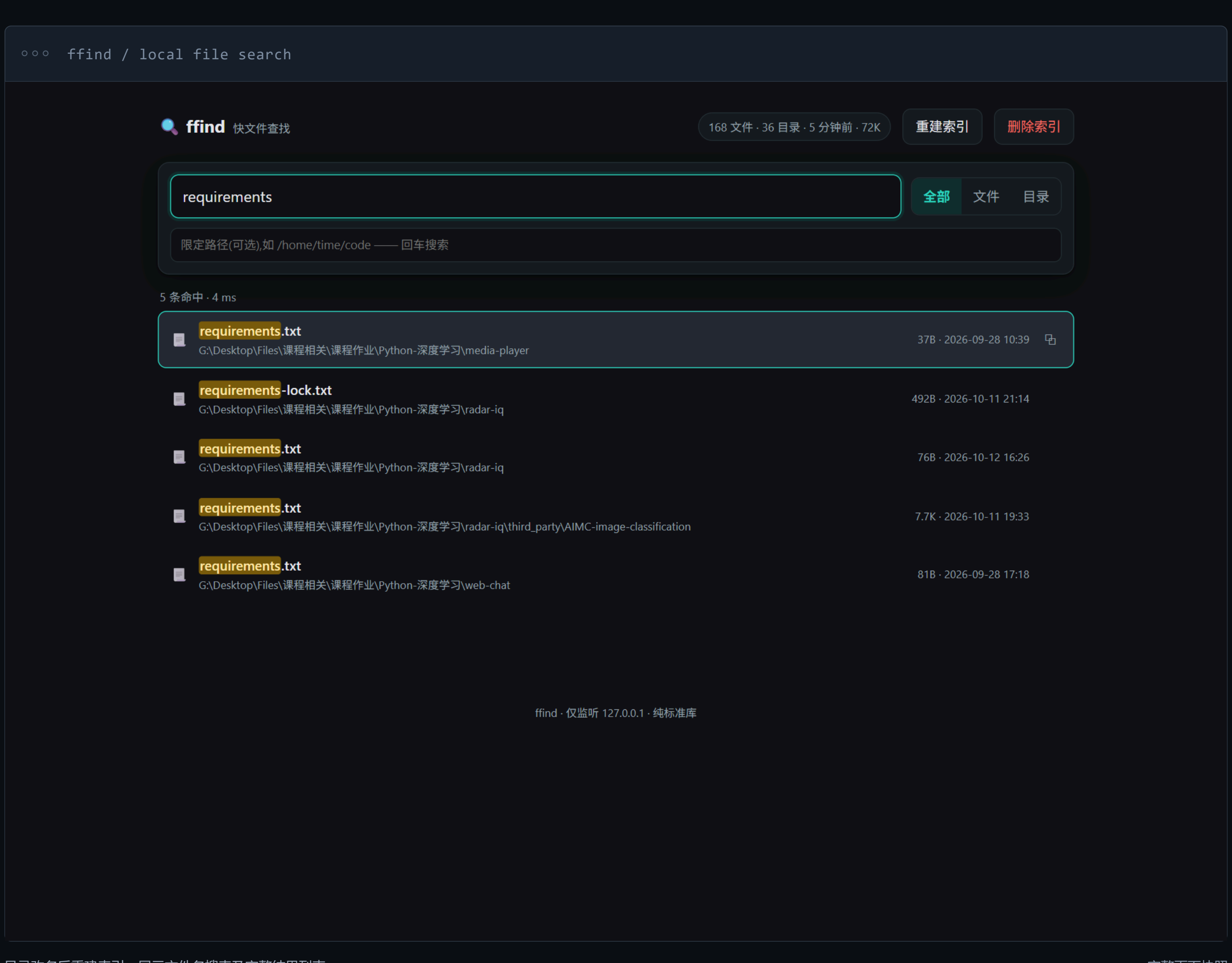
前缀查询
`name LIKE 'report%'`
利用 B-tree 的有序结构，定位到一段连续索引范围，减少需要检查的条目。

任意位置匹配
`name LIKE '%report%'`
通常仍需扫描候选条目，不能获得与前缀查询相同的范围定位加速。

> **覆盖索引的适用边界**
> 当前查询还会读取 `is_dir`，而 `(name, path)` 索引不包含该字段，因此不能保证全程只读索引、不回表。

数据库结构依据实际建表与查询代码

索引是文件名快照；新增、改名或删除后需要重建。



168 文件 · 36 目录 · 5 分钟前 · 72K

requirements

限定路径(可选),如 /home/time/code —— 回车搜索

5 条命中 · 4 ms

requirements.txt G:\Desktop\Files\课程相关\课程作业\Python-深度学习\media-player	37B · 2026-09-28 10:39
requirements-lock.txt G:\Desktop\Files\课程相关\课程作业\Python-深度学习\radar-iq	492B · 2026-10-11 21:14
requirements.txt G:\Desktop\Files\课程相关\课程作业\Python-深度学习\radar-iq	76B · 2026-10-12 16:26
requirements.txt G:\Desktop\Files\课程相关\课程作业\Python-深度学习\radar-iq\third_party\AIMC-image-classification	7.7K · 2026-10-11 19:33
requirements.txt G:\Desktop\Files\课程相关\课程作业\Python-深度学习\web-chat	81B · 2026-09-28 17:18

ffind · 仅监听 127.0.0.1 · 纯标准库

目录改名后重建索引，展示文件名搜索及完整结果列表。

完整页面快照

---

05 / AUDIO & VIDEO PLAYBACK

• Python 桌面应用

SimplePlayer 音视频播放器

基于 Tkinter 自绘界面与 FFmpeg 解码，实现音视频播放、整窗拖放与实时音频频谱。

Tkinter / Canvas　ffpyplayer　FFmpeg　tkinterdnd2

播放数据通路

五个处理阶段

01	02	03	04	05
媒体文件	ffpyplayer	PPM (P6)	PhotoImage	Canvas
文件选择 / 整窗拖放	FFmpeg 解码为 rgb24	像素附加文件头	Tk 原生解析成像	swscale 等比缩放后显示

异步状态控制

在调用入口明确状态边界

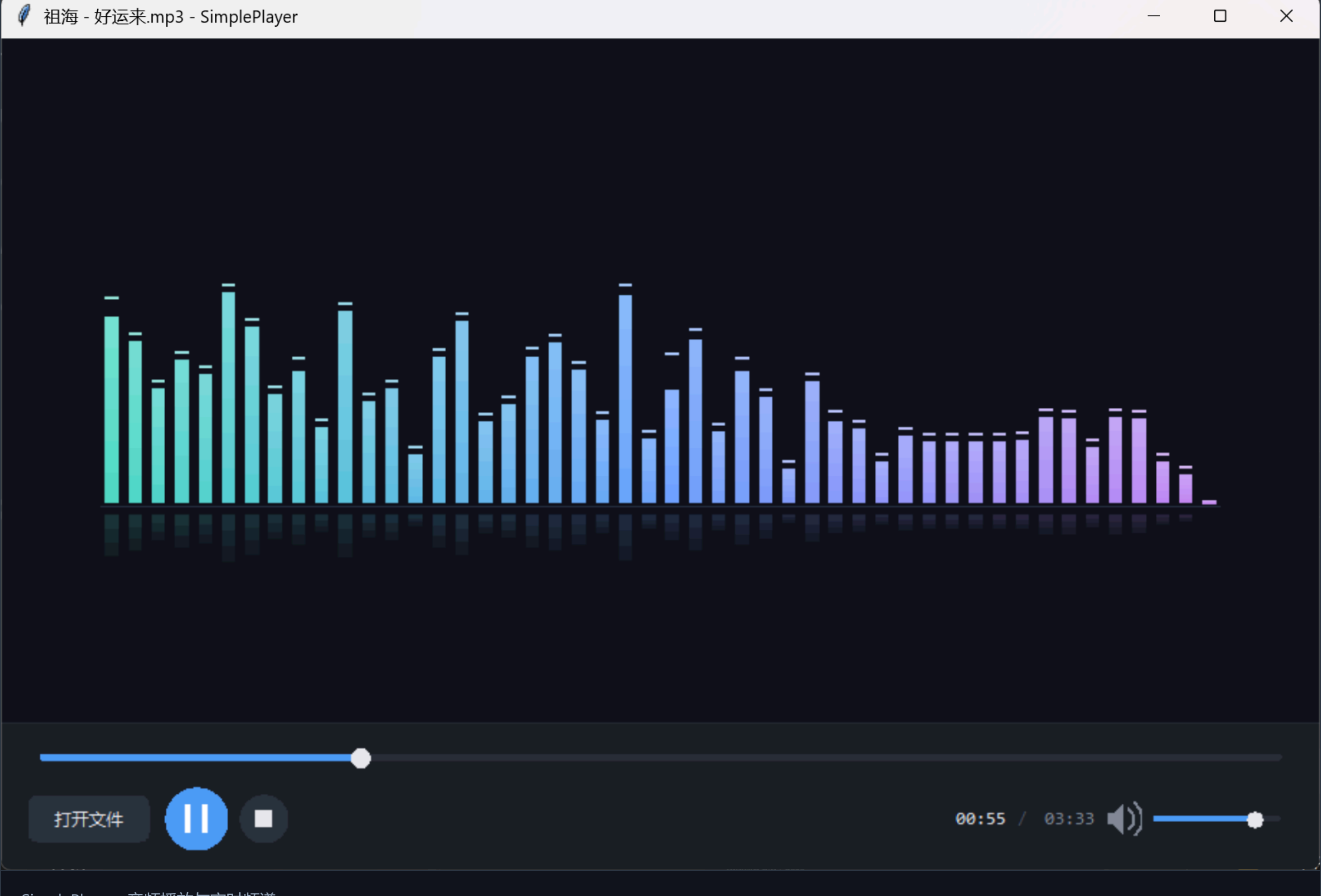
01 **就绪后读取**
_ready 根据元数据判断是否可安全取时间，避免文件尚未打开时调用底层接口。

02 **暂停定位预览**
_scrubbing 标记临时取帧过程，获取目标帧后恢复暂停，更新静止画面。

03 **以显示帧计时**
按目标时间戳丢弃旧帧，以 frame_pts 驱动进度显示，减少画面与进度错位。

Canvas 自绘按钮、图标、进度与音量控件

复用已有 FFmpeg 分析音频频谱，Canvas 绘制 48 根频段柱形。



SimplePlayer · 音频播放与实时频谱

ENGINEERING NOTES / 项目复盘

工程经验

围绕算法、数据、状态与环境，归纳五个项目的实现方法与体会。

	关注点	项目中的方法与体会
01	**明确算法分工** 贪吃蛇	DQN 估计动作的长期收益，规划器检查完整棋盘上的路线。把模型判断与安全约束分开，才能分别评估两者的作用，也能解释每一步的选择。
02	**统一数据与评估口径** 脉内调制识别	训练和推理保持相同的采样率、脉冲长度与时频处理。分别评估合成信号和官方信号，保留独立测试集；说明数据来源及实验条件，避免过度解读小样本结果。
03	**理清异步状态** 聊天室 / 播放器	切换房间时丢弃旧响应、避免重复消息；暂停、定位和切换媒体时明确状态与后台任务的归属。重点验证快速切换、重复操作和播放结束后的行为。
04	**结合实际环境验证** 文件查找器 / 播放器	优先复用已有 FFmpeg 完成解码与频谱分析。文件查找器调用系统文件管理器，并确保文件结果只打开所在目录；在实际系统中检查启动、异常反馈与资源释放。

交付经验　明确输入与状态 → 验证功能与边界 → 整理启停与复现步骤